

# Atlas Project

## Autonomous Multi-Agent Research & Report Assistant: Technical Specification

**Abstract:** Atlas is not a traditional predictive pipeline. It is an agentic AI system designed to operate with goal-directed autonomy. Given a single research question, it continuously plans, searches, reads, cross-checks, writes, and verifies its own work, producing a fully cited report bound to explicit evidence IDs.

## 1. The Agentic Paradigm vs. Static Pipelines

Most enterprise AI applications today are built as static pipelines: sequential chains of hard-coded prompts. While pipelines are predictable, they are fragile when confronted with complex, multi-faceted research tasks that require dynamic course correction.

Atlas is built on an **Agentic Architecture**. Control flow is not predetermined; it is dynamically resolved based on what the agents observe during execution. This paradigm shift enables deep research by introducing four key mechanics:

- **Decomposition:** Breaking complex queries into dependency-mapped sub-queries.
- **Tool Autonomy:** Agents deciding *when* and *which* tools to invoke (e.g., querying a vector database vs. scraping the live web).
- **The ReAct Loop:** Reasoning about findings before taking the next action, allowing the agent to self-terminate a search when sufficient evidence is gathered.
- **Reflection & Routing:** An internal critique mechanism that can reject substandard outputs and route the system back to earlier states.

## 2. Core Agent Roles & Responsibilities

Atlas distributes cognitive load across four distinct, specialized LLM agents, orchestrated via a shared state machine.

### 1. The Planner

**Role:** Query Decomposition

Takes the initial human prompt and structures it. Emits a typed list of 3-6 sub-questions with assigned priorities. It identifies dependencies, ensuring foundational questions are answered before complex synthesis is attempted.

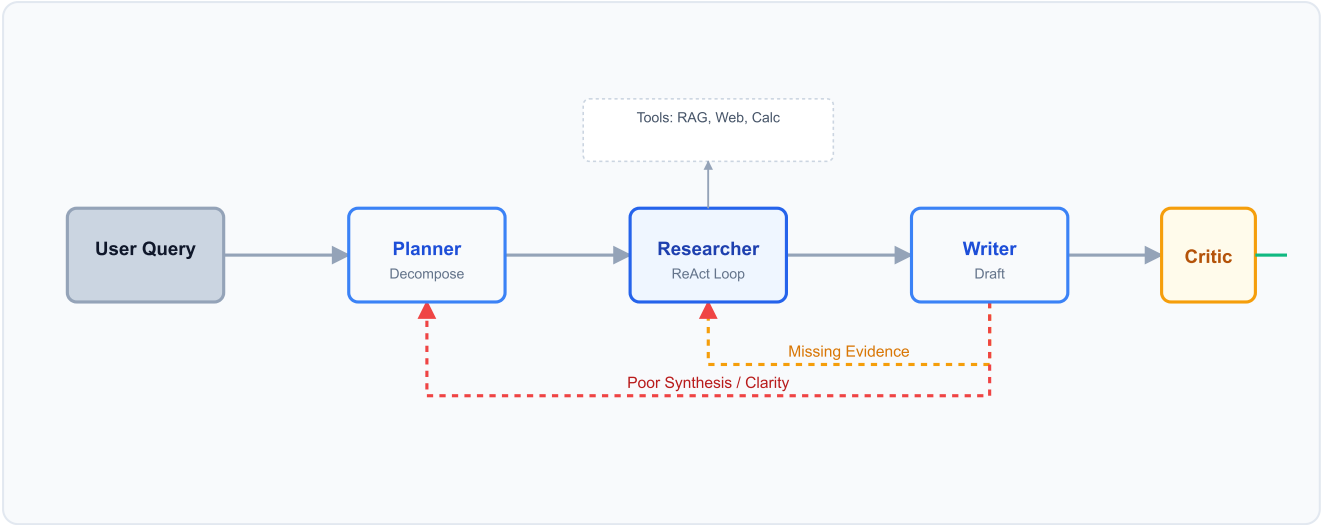
### 2. The Researcher

**Role:** Information Foraging

Executes a ReAct loop for each sub-question. It dynamically selects tools (RAG retrieval, web search, calculation) based on the context. Crucially, it stops searching only when it judges the collected evidence as sufficient.

### 3. System Architecture & Data Flow

Atlas is orchestrated using **LangGraph**, allowing for cyclic workflows and shared state memory. Unlike sequential chains, LangGraph models the system as a state machine where conditional edges govern the flow.



#### Shared State Management

A highly structured typed contract (using Pydantic models) governs the memory state passed between agents. This prevents the degradation of context that typically plagues long-running conversational LLMs.

| State Object     | Properties & Purpose                                                                                                                                      |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Plan             | sub_questions[] (List of mapped queries), priorities. Ensures the Researcher knows exactly what scope needs to be satisfied.                              |
| Evidence Arsenal | evidence[] containing: id, sub_question_id, text_snippet, source_url, retrieval_method. This explicit binding is what allows the Critic to verify claims. |
| Draft            | The Markdown text generated by the Writer, populated with inline [ID] citations corresponding to the Evidence Arsenal.                                    |
| Critic Verdict   | approved (Boolean), issues[] (Array of specific failures with location and route_to instructions).                                                        |
| Telemetry        | retry_count, tokens_used, cost_estimate. Ensures bounded autonomy by forcing early exits if budgets are exceeded.                                         |

## 4. Bounded Autonomy & Guardrails

A common failure mode of agentic AI is the "infinite loop"—an agent continually trying and failing to find information, consuming massive token budgets. Atlas implements strict **Bounded Autonomy** to ensure safe, cost-effective execution.

- **Graceful "Insufficient Evidence" Exit:** If the Researcher cannot find sources to answer a sub-question after *N* tool calls, it does not invent an answer. It logs a specific `missing_data` flag. The Writer is programmed to explicitly state in the final report: *"Insufficient verifiable data available regarding [Topic]."*
- **Retry Caps:** The overall graph has a hard iteration limit. If the Critic rejects the draft more than 3 times, the system terminates and returns the best draft alongside a warning note.
- **Untrusted Data Sandbox:** Retrieved text (especially from web searches) is treated strictly as data payloads, never as instructions. This prevents prompt injection attacks embedded in scraped web pages.
- **Cost & Token Budgets:** Telemetry is tracked natively in the `LangGraph` state. Execution halts immediately if the run approaches predefined per-report token limits.

## 5. Tool Implementation Strategy

The Researcher agent has access to a discrete set of tools. The intelligence of the system lies in how the LLM chooses which tool to invoke based on the current state of its ReAct loop.

**Retrieval First, Web Second:** To maintain domain authority and control costs, the Researcher is heavily biased (via system prompts) to invoke the `retrieve(query, k)` tool against the local Chroma vector database first. Only if the semantic search yields low-relevance scores will it fallback to the `web_search(query)` API.

| Tool Name                | Backend Tech                | Invocation Logic & Constraints                                                                                                                       |
|--------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>retrieve</code>    | Chroma DB, Local Embeddings | Queries an indexed corpus (100-300 documents). Returns top-K chunks. Extremely fast and free. Used for grounded policy/internal document queries.    |
| <code>web_search</code>  | Tavily / SerpAPI            | Pulls live web snippets. Results are cached locally by query hash to prevent duplicate billing during retry loops.                                   |
| <code>python_calc</code> | Restricted AST Evaluator    | Used when the prompt requires aggregations or math (e.g., "What is the average revenue growth?"). Operates in a highly sandboxed Python environment. |

## 6. The 4-Week Implementation Plan

To ensure a functional, portfolio-grade system within the graduation deadline, the development scope is strictly bounded. The plan implements the core agentic loop first, treating advanced memory and UI polish as secondary stretch goals.

| Phase  | Focus Area                | Key Deliverables & Milestones                                                                                                                                                                                                                                                                                                  |
|--------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Week 1 | Foundation & Single-Agent | <ul style="list-style-type: none"><li>• Lock tech stack (Python, LangGraph, Chroma).</li><li>• Ingest &amp; chunk local document corpus.</li><li>• Build standalone Researcher ReAct loop capable of basic RAG and web fallback.</li></ul> <i>Gate 1: Single agent answers questions with sources.</i>                         |
| Week 2 | Multi-Agent Orchestration | <ul style="list-style-type: none"><li>• Define LangGraph state schema &amp; edges.</li><li>• Implement Planner (structured output).</li><li>• Implement Critic and conditional routing.</li><li>• Build guardrails (retry caps, failure handling).</li></ul> <i>Gate 2: Full cycle completes on 80% of test questions.</i>     |
| Week 3 | Observability & Eval      | <ul style="list-style-type: none"><li>• Integrate tracing (Langfuse/LangSmith).</li><li>• Run automated evaluation (RAGAS framework).</li><li>• Perform 3-way ablation study (Base LLM vs RAG vs Atlas).</li><li>• Deploy Docker container to free-tier host.</li></ul> <i>Gate 3: Feature Freeze. Tracing dashboard live.</i> |
| Week 4 | Finalization & Demo       | <ul style="list-style-type: none"><li>• Document limitations and compile ablation charts.</li><li>• Build final Streamlit UI for presentation.</li><li>• Prepare backup cached runs for live demo safety.</li></ul> <i>Final Deliverable: Deployed system and presentation.</i>                                                |

## 7. Success Metrics & Evaluation

Atlas is not evaluated simply on whether it "looks impressive." It is subjected to rigorous quantitative evaluation using a 25-question test set comprising answerable, unanswerable, and contradictory scenarios.

- **Citation Precision:** Target >90%. Measured programmatically to ensure every cited [ID] exists in the evidence array, verified by human spot-checks.
- **Faithfulness & Grounding:** Evaluated using the RAGAS metric framework to ensure the Writer did not hallucinate outside the retrieved context.
- **Critic Efficacy:** The system tracks the delta between Draft 1 and the Final Draft. Success is defined as